

Triple T Engineering Journal

WRO Future Engineers 2026

Team: Triple T **Members:** Tuy Caroline, Thavrak Bunn Raksa, Lim Anita **Coach:** Lean Ratana **Country:** Cambodia **Organizer:** STEM Cambodia **Competition date:** September 5, 2026 **Documentation deadline:** August 28, 2026



Team Triple T

1. Purpose and evidence policy

This journal describes the current development state of Team Triple T's autonomous vehicle for the WRO Future Engineers competition. It combines information supplied by the team with a read-only technical inspection of the Raspberry Pi project conducted on August 28, 2026.

The report distinguishes observed facts from estimates and missing evidence. Unknown measurements and unfinished functions are stated directly instead of being replaced by invented values. This is important because the WRO rubric rewards justified decisions, recorded tests, iteration, and reproducibility rather than unsupported claims.

The inspected software project is located at `/home/aaron/robot_car` on the vehicle's Raspberry Pi. At the time of inspection, this folder was not a Git repository. The public competition repository was created separately to organize the available documentation.

2. Project overview

The current prototype is a four-wheel vehicle with front Ackermann-style steering and rear propulsion. A Raspberry Pi 5 performs camera processing and controls an L298N motor driver and MG90S steering servo. An Intel RealSense D435i provides color and depth images.

The software is written in Python. OpenCV converts camera frames into HSV color space and detects colored regions through thresholding, morphological filtering, and contour analysis. Depth information is sampled around detected objects. Separate experimental programs provide color-reactive steering and line following.

The present system is an experimental prototype, not a completed competition solution. Parking, lap counting, direction selection, finish detection, and an end-to-end challenge state machine have not yet been implemented.



Vehicle side view

3. Mobility and mechanical design

3.1 Chassis

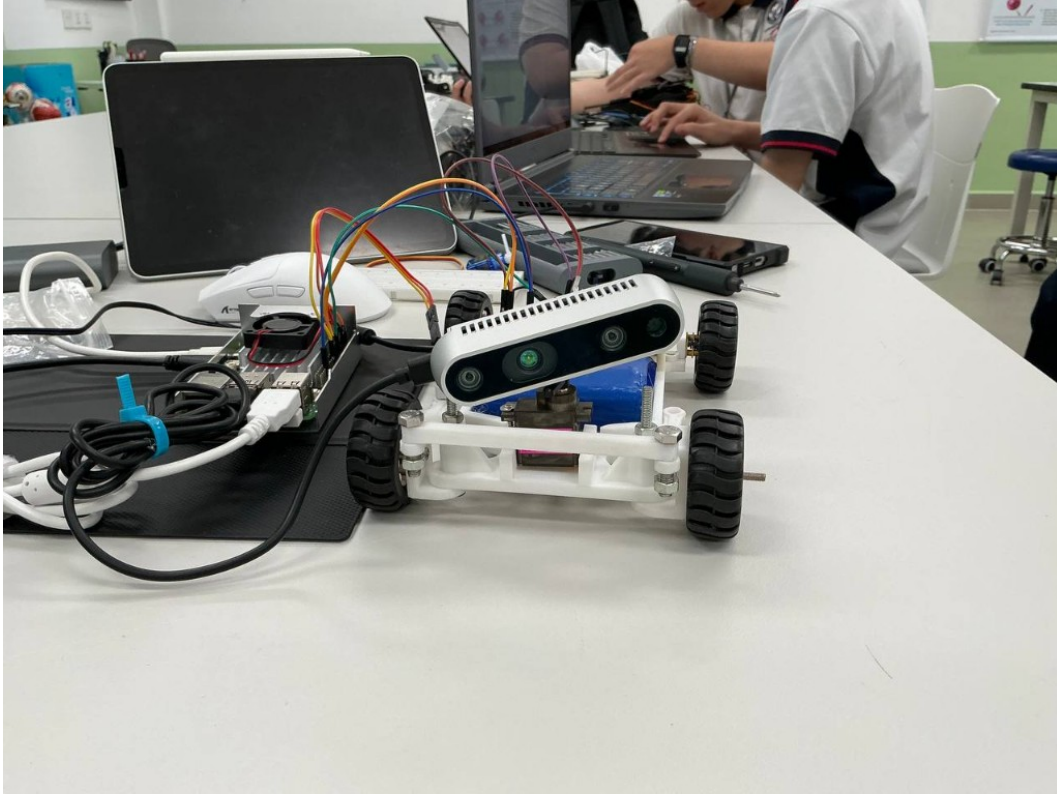
The vehicle uses an Ackermann-style chassis. The front wheels steer through a mechanical linkage moved by the steering servo. The rear wheels provide propulsion. This geometry was selected by the team before the current documentation process; no written comparison against alternative chassis designs was found, so this report does not claim a tested tradeoff that cannot be demonstrated.

Observed or team-reported components:

- Four-wheel chassis
- Front steering axle
- Rear drive axle
- MG90S steering servo reported by the team
- Two rear DC motors, with exact model not yet identified

- L298N dual H-bridge motor driver
- 12 V motor battery, with capacity not yet recorded

The robot photographs show the steering linkage, wheel layout, camera mounting, motor driver, Raspberry Pi, breadboard, and temporary wiring. The design remains in a development configuration with exposed jumper wires and a full-size breadboard.



Vehicle front view

3.2 Drive arrangement and compliance risk

The inspected notes state that both rear motors are wired in parallel to one L298N channel. Although one electrical channel gives both motors the same command, the photographs and notes do not prove that the two drive wheels are physically linked by an axle, gearbox, chain, or other mechanical connection.

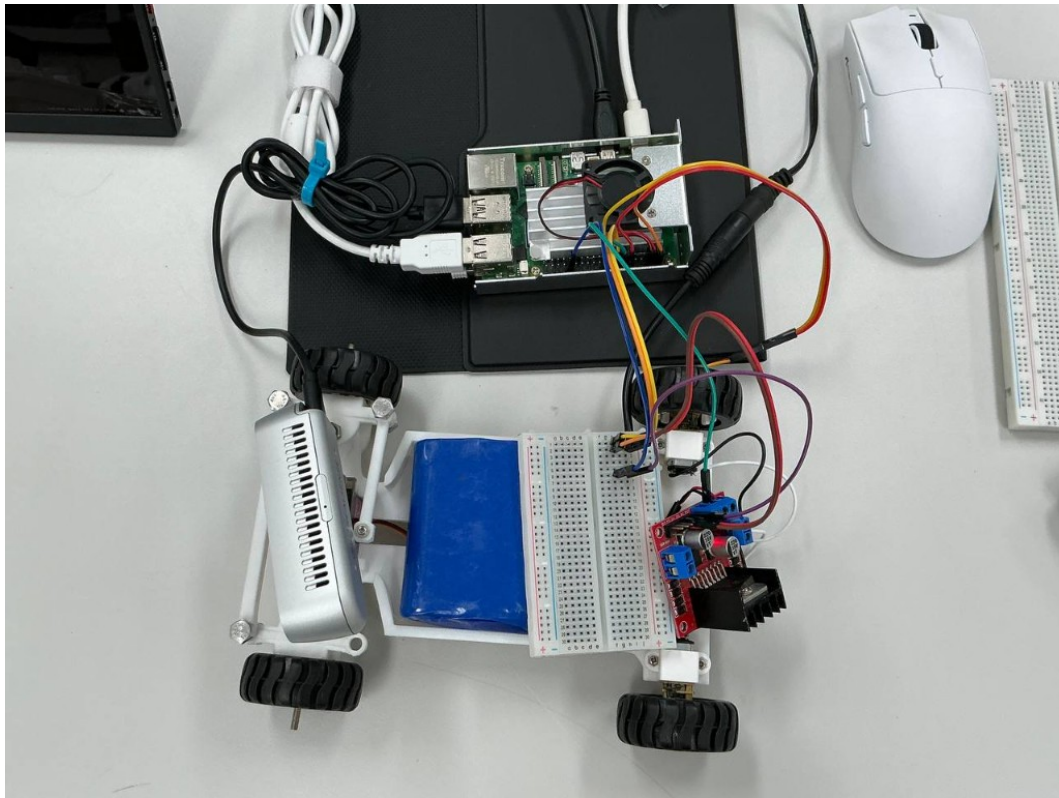
WRO rule 11.13 states that two drive motors may not independently connect to separate drive wheels. The team must verify and, if necessary, redesign this mechanism before competition. Documentation cannot resolve a physical compliance problem.

3.3 Missing measurements

The following values were not available during the inspection:

- Overall length, width, and height
- Total vehicle mass
- Wheel diameter
- Wheelbase
- Front and rear track width
- Motor model, nominal speed, torque, and stall current
- Gear ratio

Without these values, a valid torque and speed calculation cannot be produced. The team should measure and record them, then compare calculated vehicle speed with measured track speed. Any future calculation should state its assumptions and units.



Vehicle top view

4. Power and sensor architecture

4.1 Computing and sensing

The main computer was directly identified as a Raspberry Pi 5 Model B Rev 1.1 with 4 GB RAM. It runs a 64-bit Raspberry Pi operating-system image based on Debian GNU/Linux 13. The Intel RealSense D435i was identified through USB device information.

The current project uses:

- Python 3.13.5
- OpenCV 4.10.0
- NumPy 2.2.4
- gpiozero 2.0.1
- I2C 0.2.2.0
- V4L2/UVC camera access

The RealSense color and depth streams are configured at 640 by 480 pixels. The IMU is not used by the present software. The Python RealSense library is not installed, so the project uses standard Linux video devices and a custom V4L2 depth reader.

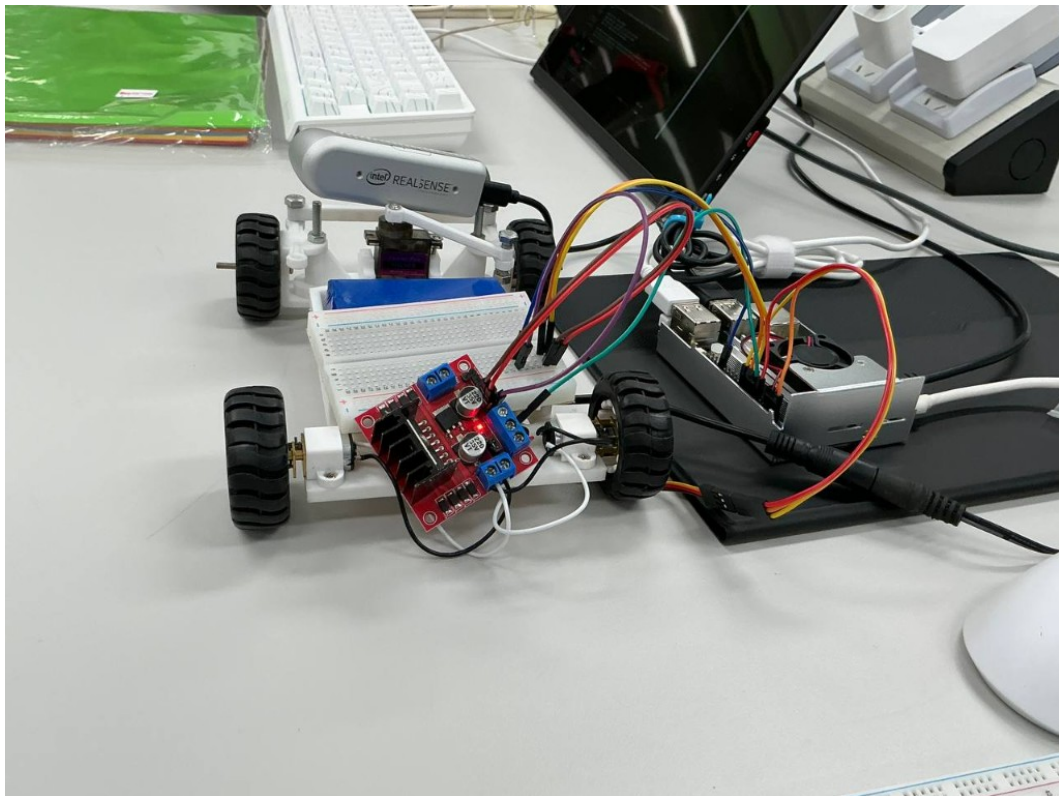
4.2 GPIO and wiring

The project notes and source-code constants agree on this GPIO map:

- GPIO18, physical pin 12: steering-servo signal

- GPIO23, physical pin 16: L298N IN1
- GPIO24, physical pin 18: L298N IN2
- GPIO12, physical pin 32: L298N ENA
- Physical pin 2: 5 V connection for the steering servo
- Physical pin 14: common ground to the motor driver
- Physical pin 20: steering-servo ground
- USB: RealSense D435i connection

The battery positive terminal connects to the L298N motor-power input. Battery ground, L298N ground, and Raspberry Pi ground must share a common reference. The exact wiring must be checked physically against this documentation before operation.



Vehicle electronics view

4.3 Power limitation

During development, the Raspberry Pi uses a separate USB-C supply. The available 12 V battery supplies the motor system. A portable regulated supply for the Raspberry Pi has not yet been selected.

The Raspberry Pi must never be connected directly to 12 V. It requires a suitable regulated 5 V supply through an appropriate power input. The RealSense camera, Raspberry Pi, cooling fan, servo, and motors create different load conditions, so the final design needs measured normal and peak current values.

No battery capacity, regulator specification, motor current, or measured system-power data were available. Therefore, this report does not present estimated values as a verified power budget. Required measurements include:

- Raspberry Pi and camera idle current
- Processing current while the camera and control loop run
- Servo current during steering and at stall

- Motor current with the wheels raised, on the field, and stalled
- Battery voltage before and after repeated runs
- Runtime under representative competition conditions

5. Camera perception

5.1 Color processing

The shared `color_detection.py` module converts BGR images to OpenCV HSV. It creates color masks, applies a 5 by 5 morphological opening, applies a 9 by 9 morphological closing, and extracts external contours. Contours with an area of 300 pixels or less are rejected.

The observed color settings include:

- Red: hue center 178 with tolerance 10
- Green: hue center 59 with tolerance 20
- Blue: hue center 110 with tolerance 15
- Dark blue: hue center 115 with tolerance 15

Saturation and value limits are also applied. Red includes hue wrap-around near zero. Black is treated mainly as a low-brightness range.

The calibration tools provide trackbars and sampling programs so thresholds can be changed under venue lighting. Automatic white balance and exposure are enabled through camera controls. However, no saved calibration records were found, and no precision or false-positive measurements have been recorded.

5.2 Depth processing

The project contains a custom depth reader because OpenCV could not directly open the RealSense Z16 depth stream. The implementation uses V4L2, memory mapping, and a background reading thread.

For a detected object's center, the program maps the color-image coordinate into the depth frame and takes the median valid depth from an 11 by 11 region. Median sampling helps reduce the influence of isolated invalid pixels. The current implementation uses proportional coordinate scaling rather than a documented RealSense color-to-depth alignment calibration.

5.3 Known perception risks

- Color decisions are made frame by frame without temporal filtering.
- Strong lighting changes and shadows may produce false detections.
- The green thresholds differ between the shared detector and an older human-detection file.
- The current driving code relies on valid depth for its distance checks.
- Loss of the depth stream can prevent expected stop behavior.
- No measured detection accuracy or processing frame rate is available.

6. Software architecture and behavior

6.1 Main files

The inspected project contains:

- `steer_by_color.py`: reactive color-based steering and stop experiment

- `line_follow.py`: border-region line-following experiment
- `color_detection.py`: reusable HSV detection and depth lookup
- `depth_camera.py`: custom V4L2 Z16 depth reader
- `color_probe.py` and `color_sample.py`: calibration helpers
- `test_car.py` and `test_car_gui.py`: hardware tests
- `drive_terminal.py`: manual control for bring-up
- `drive_by_color.py`: simple color-to-motion demonstration
- `human_detection.py`: experimental person detection, not connected to driving
- `autonomous_drive.py`: corrupt file containing NUL bytes and not usable

6.2 Color-steering execution

The current color-steering program performs these steps:

1. Open a 640 by 480 color stream.
2. Configure automatic color controls.
3. Start a background reader for the 640 by 480 depth stream.
4. Detect red, green, and dark-blue contours in each frame.
5. Find the nearest valid depth for each relevant color.
6. Drive forward or stop and set a fixed steering angle.
7. Draw the detections for development feedback.
8. Stop the motor, center the servo, and release resources during shutdown.

The stop threshold is 500 mm. Steering and motor decisions are recalculated for each frame. The program does not contain a competition-level state machine.

6.3 Line-following execution

The line-following experiment creates a combined dark-blue and black mask. It examines vertical strips 80 pixels wide at both image edges. If both strips see enough matching pixels, the car continues straight. If one side loses the detected line, the program turns toward that side. The required matching fraction is 3% of the strip.

This is fixed-angle, bang-bang steering. No proportional, integral, or derivative controller was found. The motor is either commanded forward or stopped; the PWM-capable enable connection is not currently used for proportional speed control.

6.4 Safety and edge cases

Positive safety evidence includes cleanup logic that stops the motor and centers the steering servo when a loop exits. Current unresolved edge cases include:

- Simultaneous red and green detection gives red priority because it is checked first.
- Missing depth can prevent all distance-gated actions.
- Servo-angle conventions differ between some manual and autonomous scripts.
- There is no validated behavior for camera loss.
- There is no lap counting, direction mode, finish detection, or parking state.

7. Challenge strategy status

7.1 Open Challenge

A complete Open Challenge strategy is not present. The line-following experiment may contribute to track navigation, but the code does not count sections or laps, choose clockwise versus counterclockwise travel, recognize the finish section, or stop after three laps.

7.2 Obstacle Challenge

Red and green color detection exists, and the color-steering experiment maps detections to fixed steering commands. This is not yet a complete side-obedience strategy. The code does not manage multiple pillars, section transitions, three official laps, the post-lap route to parking, or randomized starting conditions.

7.3 Parking

Parking is not implemented. No parking detector, state, trajectory, reverse movement, or final alignment check was found.

8. Engineering decisions and iteration

The available code provides evidence of technical iteration even though no dated engineering journal existed on the robot:

- Camera access supports a fallback because the preferred RealSense Python library was unavailable.
- A custom depth reader addresses the inability of OpenCV to open the depth stream directly.
- Color-probe and color-sample tools support threshold tuning.
- Morphological filters and contour-area limits reduce small noisy detections.
- Median depth sampling rejects invalid zero-depth pixels.
- Separate line-following and color-steering programs explore different navigation approaches.
- Different fixed angles appear in the two experiments, indicating behavior-specific tuning.

These observations show changes in the software, but they do not prove which team member made each decision, when it occurred, which alternatives were tested, or how much performance improved. Those claims require team notes, dated commits, or measured test records.

9. Testing and results

Hardware test programs exist for servo movement and motor forward/backward operation. Calibration programs exist for HSV sampling. Compiled Python files show that several modules have been executed or imported.

No dated logs, datasets, autonomous-run videos, measured lap results, detection accuracy, frame-rate measurements, current measurements, or recorded calibration outputs were found. Consequently, the project currently has test tools but limited test evidence.

A useful repeatable test record should include:

- Date and software version
- Test objective
- Field and lighting conditions
- Number of attempts
- Success and failure definition

- Raw observations
- Measured result
- Change made afterward
- Result after the change

The team should avoid claiming improvement without before-and-after measurements.

10. Reproduction instructions

The current prototype can be started from the existing Raspberry Pi environment with:

```
cd /home/aaron/robot_car
robot_env/bin/python steer_by_color.py
robot_env/bin/python line_follow.py
robot_env/bin/python color_detection.py
```

Before running any movement test:

1. Raise the drive wheels or place the vehicle in a safe test area.
2. Verify motor polarity and steering direction.
3. Verify the common ground.
4. Confirm the Raspberry Pi receives regulated power, never 12 V.
5. Confirm the camera devices are available.
6. Run the hardware tests at low risk.
7. Calibrate colors for the current lighting.
8. Keep a manual method available to disconnect motor power.

Full reproduction is not yet possible because the public repository does not contain the inspected Python source files, dependency manifest, exact motor data, CAD, dimensioned drawings, or measured calibration and test data.

11. Current limitations and required next work

The following items are incomplete and must not be represented as finished:

- Transfer and organize the Python source code in GitHub.
- Replace or remove the corrupt `autonomous_drive.py`.
- Confirm the steering-servo model discrepancy.
- Identify the drive motors and battery capacity.
- Measure the complete vehicle.
- Resolve the drive-system compliance risk.
- Select a portable regulated Raspberry Pi supply.
- Create and verify a power budget.
- Implement a competition state machine.
- Implement direction handling, lap counting, and finish detection.
- Complete obstacle side-obedience logic.
- Implement and test parking.
- Record quantitative tests and improvements.

- Produce CAD or dimensioned mechanical drawings.
- Add a bottom-view photograph.
- Record public or unlisted autonomous videos for both challenges.

12. Evidence references

This journal is supported by:

- The public GitHub README and vehicle photographs
- The complete read-only robot audit in docs/WRO_ROBOT_AUDIT.md
- The source-code inventory observed at /home/aaron/robot_car
- WRO Future Engineers Game Rules 2026

The engineering journal will be updated when additional measurements, source files, test results, diagrams, and videos are supplied.